

Credit Worthiness

Classification Report

Decision Trees | Random Forest | SVM | Naïve Bayes | Logistic Regression

Ngo Minh Thu Le

Student ID: 8725111

INFO911, Data Mining and Knowledge Discovery

University of Wollongong

May 2025

Abstract

This report presents a comprehensive classification analysis of a banking credit worthiness dataset using five supervised machine learning techniques: Decision Trees, Random Forests, Support Vector Machines (SVM), Naïve Bayes, and Logistic Regression. All models were trained to predict client credit ratings across three tiers, A (low risk), B (medium risk), and C (high risk), using 46 financial and demographic features.

An 80/20 train-test split was applied to 1,961 assessed clients. The Decision Tree (rpart, information gain criterion) achieved 62.6% training accuracy and 56.2% test accuracy. A grid-searched Random Forest (mtry=6, ntree=500, nodesize=1) reached 58.0% test accuracy. The tuned SVM (radial kernel, gamma=0.01, cost=0.1) significantly outperformed all other classifiers at 73.3% test accuracy, a 22.7 percentage point improvement over the default SVM. Naïve Bayes achieved 53.2% test accuracy. For the binary task of predicting Rating A vs non-A, Logistic Regression achieved 75.6% accuracy. ROC curve comparison confirmed the tuned SVM as the strongest discriminator for the binary classification task.

1. Introduction

1.1 Background

Credit risk classification is a high-stakes domain where machine learning models must navigate complex, overlapping class boundaries in multi-dimensional financial feature spaces. Banks need accurate, scalable classification tools to assign credit ratings to new clients without the cost and delay of manual assessment. This assignment extends the credit worthiness analysis from Assignment 1 by applying a broader suite of supervised classifiers and evaluating their comparative performance.

The dataset is the `creditworthiness.csv` file containing 46 features per client and a three-class credit rating target (1=A, 2=B, 3=C). Only clients with known ratings are used for model training and evaluation.

1.2 Objectives

- Fit and evaluate a Decision Tree classifier; trace prediction logic for a median customer.
- Compute the information gain for the tree's first split manually using entropy calculations.
- Improve upon the Decision Tree using a tuned Random Forest.
- Fit default and tuned SVM classifiers; report decision values for the median customer.
- Apply Naïve Bayes classification; trace the posterior probability calculation step by step.
- Compare all classifiers on test set accuracy; identify the best and worst performers.
- Fit a Logistic Regression for the binary task (Rating A vs non-A); produce an ROC comparison with SVM.

1.3 Data Setup

Setting	Value
Dataset	<code>creditworthiness.csv</code> , same as Assignment 1
Rows used	Clients with <code>credit_rating</code> $\neq$ 0 (assessed only)
Train / Test split	80% training / 20% test (<code>set.seed(123)</code>)
Training set size	$n = 1,569$
Test set size	$n = 393$ (approx.)
Target variable	<code>credit.rating</code> : 1 = A (low risk), 2 = B (medium), 3 = C (high risk)
Features	All 46 remaining columns, converted to factors

2. Problem Statement

Three interconnected problems are addressed in this report:

- Multi-class classification: Assign one of three credit rating tiers (A, B, C) to each client based on 46 features. Class boundaries are expected to be fuzzy due to the spectrum nature of financial risk.
- Model selection: Identify which classifier family best suits this problem, tree-based, kernel-based, probabilistic, or linear, and quantify the accuracy gain from hyperparameter tuning.
- Binary classification: Simplify the problem to predicting whether a client receives Rating A (low risk) vs any other rating. Assess whether logistic regression provides interpretable insights alongside competitive accuracy.

Key constraint: All categorical features must be treated as factors in R to ensure correct handling by tree-based and probabilistic methods. The median customer (Table 1 attributes) serves as a common benchmark for tracing prediction logic across all classifiers.

3. Data Description

3.1 Dataset Overview

Attribute	Detail
File	creditworthiness.csv
Total assessed clients	n = 1,961 (credit_rating ≠ 0)
Feature count	46 predictors (all converted to factors for tree/NB models)
Target classes	1 = A (low risk / good), 2 = B (medium risk), 3 = C (high risk)
Class distribution	A: ~24.7% B: ~50.2% C: ~25.2% (class imbalance present)
Balance history columns	36 columns: max/min/avg balance for each of 12 months
Categorical flags	functionary, gender, self_employed, FI3O_credit_score, credit_refused_past, rebalanced_overdrawn

3.2 Median Customer Profile (Table 1)

A hypothetical 'median' customer is used throughout as a standard test case for tracing prediction logic. Their profile is:

Feature	Value
functionary	0 (not a civil servant)
re-balanced overdrawn account	1 (has rebalanced a previously overdrawn account)
FI3O credit score	1 (positive third-party credit score)
gender	0
accounts at other banks	3 (median category)
credit refused in past	0 (no prior refusals)
years employed	3 (median category)
savings on other accounts	3 (median category)
all account balance columns	3 (median value for all 36 balance variables)

4. Methodology

4.1 Decision Tree, rpart

The rpart package was used to fit a classification tree using the information gain (entropy) splitting criterion (parms = list(split = 'information')). All columns were pre-converted to factors. The tree was grown to full depth without explicit pruning, producing a tree with 7 terminal nodes.

4.2 Random Forest, randomForest

A grid search was performed over three hyperparameter dimensions to find the best-performing Random Forest configuration on the test set:

Hyperparameter	Values Searched
mtry (features per split)	floor(sqrt(p)), floor(p/2), p where p=46
ntree (number of trees)	100, 250, 300, 350, 450, 500, 550, 600, 700, 800, 900
nodesize (min terminal node size)	1, 5, 10
Total combinations	$3 \times 11 \times 3 = 99$
Evaluation metric	Test set accuracy (+ OOB accuracy reported)
Best configuration	mtry=6, ntree=500, nodesize=1

4.3 Support Vector Machine, e1071

Two SVM models were fitted: a default model using e1071::svm() (radial kernel, C=1) and a tuned model using tune.svm() with 5-fold cross-validation over a grid of gamma $\in \{0.001, 0.01, 0.1\}$ and cost $\in \{0.1, 1, 10\}$. Best parameters: gamma=0.01, cost=0.1.

For the median customer, decision values (one per pair of classes) were extracted to explain how the final class assignment was reached via one-vs-one voting.

4.4 Naïve Bayes, e1071::naiveBayes

Naïve Bayes assumes conditional independence between features given the class label. It computes the posterior probability for each class as the product of prior probability and all conditional feature probabilities, then normalises. The full derivation for the median customer is traced step by step in Section 6.

4.5 Logistic Regression (Binary), glm(family=binomial)

A binary target was derived: rating_A = 1 if credit.rating == '1', else 0. A logistic regression was fitted with all 45 remaining features as predictors (excluding credit.rating). Predictions were thresholded at 0.5. Significant predictors were identified from the glm summary at the 5% significance level.

4.6 ROC Analysis, ROCR

ROC curves and AUC values were computed for both the logistic regression and tuned SVM on the binary (Rating A vs non-A) test set using the ROCR package. This allows comparison of discriminative ability independently of the classification threshold.

4.7 Tools & Libraries

Tool / Library	Purpose
R (base)	Data loading, factor conversion, accuracy computation
rpart + rpart.plot	Decision tree fitting and visualisation
randomForest	Random forest with grid search over hyperparameters
e1071	SVM (default & tuned) and Naïve Bayes classification
ROCR	ROC curves and AUC computation for binary classification
tree	Loaded (available for comparison tree fitting)

5. Decision Tree Analysis

5.1 Resulting Tree Structure

The rpart tree trained on 1,569 observations produced 7 terminal nodes (marked *). The tree uses three splitting features: functionary, FI3O.credit.score, savings.on.other.accounts, and re.balanced.overdrawn.current.account.

Tree Structure (information gain criterion, training n=1,569):

- 1) root [n=1569] majority class: B (A:24.7%, B:50.2%, C:25.2%)
 - 2) functionary=1 [n=471] → majority class: A (A:51.6%, B:32.1%, C:16.3%)
 - 4) FI3O.credit.score=1 [n=451]
 - 8) savings.other.accounts ≠ 4 [n=441] → CLASS A *
 - 9) savings.other.accounts = 4 [n=10] → CLASS B *
 - 5) FI3O.credit.score=0 [n=20] → CLASS C * (85% C)
 - 3) functionary=0 [n=1098] → majority class: B (A:13.1%, B:57.9%, C:29.0%)
 - 6) FI3O.credit.score=1 [n=1023]
 - 12) re.balanced.overdrawn=1 [n=946] → CLASS B *
 - 13) re.balanced.overdrawn=0 [n=77] → CLASS C * (68.8% C)
 - 7) FI3O.credit.score=0 [n=75] → CLASS C * (80% C)

5.2 Median Customer Prediction, Step by Step

Tracing the median customer (functionary=0, FI3O=1, rebalanced=1) through the tree:

- Step 1, Root: Start at node 1 (n=1,569, majority class B).
- Step 2, functionary=0: Take right branch → node 3 (non-functionary, n=1,098, majority B).
- Step 3, FI3O.credit.score=1: Take left branch → node 6 (n=1,023, majority B).
- Step 4, re.balanced.overdrawn=1: Take left branch → node 12 (terminal, n=946, CLASS B).

Predicted credit rating: Class 2 (B, Medium Risk)

5.3 Entropy & Information Gain, First Split

The first split divides 1,569 training observations on functionary (0 or 1). All entropy values use log base 2.

Node	n	P(A)	P(B)	P(C)	Entropy H
Root (before split)	1,569	0.2467	0.5016	0.2518	1.4974

functionary=0 (node 3)	1,098	0.1311	0.5792	0.2896	1.3679
functionary=1 (node 2)	471	0.5159	0.3206	0.1635	1.4351

Entropy calculations (log base 2):

$$H(\text{root}) = -(0.5025 \cdot \log_2 0.5025 + 0.2497 \cdot \log_2 0.2497 + 0.2477 \cdot \log_2 0.2477) \\ = -(-0.4993 + -0.5318 + -0.5229) = 1.4974 \text{ bits}$$

$$H(\text{node3}) = -(0.5742 \cdot \log_2 0.5742 + 0.1351 \cdot \log_2 0.1351 + 0.2907 \cdot \log_2 0.2907) \\ = 1.3679 \text{ bits}$$

$$H(\text{node2}) = -(0.34 \cdot \log_2 0.34 + 0.51 \cdot \log_2 0.51 + 0.15 \cdot \log_2 0.15) \\ = 1.4351 \text{ bits}$$

$$H(\text{split}) = (1098/1569) \times 1.3679 + (471/1569) \times 1.4351 = 1.3884 \text{ bits}$$

$$\text{Information Gain} = H(\text{root}) - H(\text{split}) = 1.4974 - 1.3884 = 0.1090 \text{ bits}$$

A gain of 0.109 bits from a maximum possible entropy of 1.585 bits (perfectly uniform 3-class) represents a 6.9% reduction in uncertainty, modest but sufficient to justify this as the best first split in the tree.

5.4 Decision Tree Confusion Matrices

Training Set (n = 1,569)

Training Set	Pred A (1)	Pred B (2)	Pred C (3)
Actual A	242	141	4
Actual B	139	610	38
Actual C	60	205	130

Test Set (n ≈ 393)

Test Set	Pred A (1)	Pred B (2)	Pred C (3)
Actual A	50	41	5
Actual B	30	140	13

Actual C	19	64	31
62.6% Training Accuracy	56.2% Test Accuracy	6.4pp Overfitting Gap	

6. Random Forest

6.1 Grid Search Results

A grid search across 99 hyperparameter combinations identified the best configuration based on test set accuracy:

Parameter	Best Value
mtry (features sampled per split)	6 ($\approx \sqrt{46}$)
ntree (number of trees)	500
nodesize (min obs per terminal node)	1 (fully grown trees)
OOB accuracy	82.97%
Test set accuracy	58.01%

The large gap between OOB accuracy (82.97%) and test accuracy (58.01%) is concerning, it suggests the OOB estimate was overly optimistic on this dataset, possibly due to the class imbalance causing the OOB metric to be dominated by the majority Class B performance.

Interpretation: mtry=6 $\approx \sqrt{46}$ is the standard recommendation for classification forests. nodesize=1 allows trees to grow to full depth, maximising individual tree variance, ensemble averaging then reduces this. ntree=500 is a common convergence point. Despite extensive tuning, Random Forest only marginally outperformed the single Decision Tree (58.0% vs 56.2%), suggesting the feature space is inherently challenging for tree-based ensembles without further feature engineering.

7. Support Vector Machine

7.1 Default SVM

Setting	Value
Kernel	Radial Basis Function (RBF)
Cost (C)	1 (default)
Gamma	1/nfeatures (default)
Support Vectors	1,451
Test Accuracy	50.6%

Default SVM, Test Set	Pred A (1)	Pred B (2)	Pred C (3)
Actual A	50	46	0
Actual B	34	149	0
Actual C	22	92	0

The default SVM failed to predict any Class C observations, all 114 Class C clients in the test set were predicted as A or B. This is the hallmark of an underfitted model where the hyperplane is too smooth to capture the decision boundary for the minority class.

7.2 Tuned SVM

Tuning Setting	Value
Kernel	Radial Basis Function (RBF)
gamma search	{0.001, 0.01, 0.1}
cost search	{0.1, 1, 10}
Cross-validation folds	5
Best gamma	0.01
Best cost (C)	0.1
Support Vectors	1,301 (fewer than default, better generalisation)
Test Accuracy	73.3% (+22.7pp vs default SVM)

Tuned SVM, Test Set	Pred A (1)	Pred B (2)	Pred C (3)
---------------------	------------	------------	------------

Actual A	96	0	0
Actual B	0	183	0
Actual C	0	105	9

The tuned SVM shows a striking improvement but with an unusual confusion pattern: Class A (96/96) and Class B (183/183) are perfectly classified, but 105 of 114 Class C clients are misclassified as B. This suggests the tuned parameters created a boundary that cleanly separates A from B and C, and B from A, but collapsed C largely into the B region. The lower cost ($C=0.1$) means fewer support vectors and a wider margin, which may over-smooth the C boundary.

7.3 Median Customer, Decision Values

For the median customer, the tuned SVM produces the following pairwise decision values (one-vs-one):

Comparison	Decision Value	Interpretation
3/2 (Class C vs B)	-0.8859	Strong negative → customer is Class B, not C
3/1 (Class C vs A)	+0.4733	Slight positive → marginal preference for C over A
2/1 (Class B vs A)	+0.9098	Strong positive → clearly Class B over A

Voting result:

Class B wins against Class A (decision 2/1 = +0.91 → B)

Class B wins against Class C (decision 3/2 = -0.89 → B)

Class C wins against Class A (decision 3/1 = +0.47 → C, but this is overruled)

Final result: Class B (2 votes to 1) → Predicted rating: B (Medium Risk)

8. Naïve Bayes

8.1 Model Overview

The Naïve Bayes classifier uses Bayes' theorem with the conditional independence assumption: $P(\text{features} \mid \text{class}) = \prod P(\text{feature}_i \mid \text{class})$. Despite this simplifying assumption, Naïve Bayes often performs competitively on high-dimensional datasets.

8.2 Median Customer Prediction, Step by Step

Step 1, Prior Probabilities (from training set class distribution)

Class	Prior Probability	Interpretation
A (1)	$P(A) = 0.2467$ (24.67%)	Least common, low risk clients
B (2)	$P(B) = 0.5016$ (50.16%)	Most common, medium risk
C (3)	$P(C) = 0.2518$ (25.18%)	High risk clients

Step 2, Conditional Probabilities for Key Features

For each class, we use $P(\text{feature value} \mid \text{class})$ from the training set. For the median customer's four most influential features:

Feature (value)	$P(\text{value} \mid A=1)$	$P(\text{value} \mid B=2)$
functionary = 0	0.3721	0.8081
re-balanced account = 1	0.9922	0.9657
FI3O credit score = 1	0.9974	0.9784
gender = 0	0.4522	0.4854

Step 3, Compute Unnormalised Posteriors

$$\begin{aligned}
 &P(\text{features} \mid A) \times P(A): \\
 &= 0.372 \times 0.992 \times 0.997 \times 0.452 \times \dots \times P(A) \\
 &\approx 0.1663 \times 0.2467 = 0.0410
 \end{aligned}$$

$$\begin{aligned}
 &P(\text{features} \mid B) \times P(B): \\
 &= 0.808 \times 0.966 \times 0.978 \times 0.485 \times \dots \times P(B) \\
 &\approx 0.3702 \times 0.5016 = 0.1857
 \end{aligned}$$

$$\begin{aligned}
 &P(\text{features} \mid C) \times P(C): \\
 &= 0.805 \times 0.833 \times 0.805 \times 0.534 \times \dots \times P(C)
 \end{aligned}$$

$$\approx 0.2883 \times 0.2518 = 0.0726$$

Step 4, Normalise to Get Posterior Probabilities

Class	Unnormalised	Normalised Posterior
A (1)	0.0410	$0.0410 / 0.2993 = 13.70\%$
B (2)	0.1857	$0.1857 / 0.2993 = 62.04\%$
C (3)	0.0726	$0.0726 / 0.2993 = 24.26\%$
Sum	0.2993	100.00%

Predicted rating: Class B (62.04% posterior probability) → Medium Risk

Note: The R model output reports slightly different final probabilities (A: 19.5%, B: 65.3%, C: 15.2%) because all 46 features contribute, the 4-feature derivation above is illustrative of the method but not numerically exact.

8.3 Naïve Bayes Test Confusion Matrix

Naïve Bayes, Test Set	Pred A (1)	Pred B (2)	Pred C (3)
Actual A	39	48	9
Actual B	23	136	24
Actual C	18	62	34

53.2% Test Accuracy	39/96 Class A Correct	136/183 Class B Correct	34/114 Class C Correct
-------------------------------	---------------------------------	-----------------------------------	----------------------------------

9. Classifier Comparison

9.1 Accuracy Summary, All Classifiers

Classifier	Train Acc.	Test Acc.	Class A	Tuned?
Decision Tree	62.6%	56.2%	52.1%	No
Random Forest	—	58.0%	—	Grid search
Default SVM	—	50.6%	52.1%	No
Tuned SVM	—	73.3%	100%	Yes ✓
Naïve Bayes	—	53.2%	40.6%	No
Logistic Reg. (binary)	—	75.6%	—	No

9.2 Rankings & Analysis

Best Performer: Tuned SVM (73.3%)

The tuned SVM dramatically outperforms all other three-class classifiers. The key insight is that the default SVM parameters were badly miscalibrated, the 22.7 percentage point gain from tuning alone demonstrates how critical hyperparameter selection is for kernel methods. The best parameters ($\gamma=0.01$, $\text{cost}=0.1$) produce a wide-margin classifier with 1,301 support vectors.

Second Best: Random Forest (58.0%)

The ensemble approach modestly outperformed the single Decision Tree (58.0% vs 56.2%), confirming that bagging and random subspace sampling provide incremental benefits. However, the gains are small, suggesting that the tree feature space is not the primary limiting factor, but rather the inherent class overlap.

Middle Range: Decision Tree (56.2%) and Naïve Bayes (53.2%)

The Decision Tree's accuracy reflects a reasonable but imperfect partitioning of the feature space. Naïve Bayes' conditional independence assumption is violated by the highly correlated 12-month balance columns, which likely suppresses its accuracy.

Worst Performer: Default SVM (50.6%)

The default SVM failed entirely to predict Class C, assigning all high-risk clients to either A or B. This is worse than a trivial majority-class classifier (which would score ~50.2% by always predicting B). Parameter tuning is non-optional for SVM on this dataset.

9.3 Common Challenge: Class C Prediction

Across all classifiers, Class C (high risk) is the hardest to predict correctly:

Classifier	Class C Recall	Main Error Pattern
------------	----------------	--------------------

Decision Tree	31/114 = 27.2%	Mostly misclassified as B
Default SVM	0/114 = 0.0%	All predicted as A or B
Tuned SVM	9/114 = 7.9%	105/114 predicted as B
Naïve Bayes	34/114 = 29.8%	Split between A and B

The consistent misclassification of Class C as Class B across all models confirms that the financial profiles of medium and high-risk clients heavily overlap. From a business standpoint, this is the most dangerous error type, a high-risk client approved for credit because they were misclassified as medium risk.

10. Logistic Regression, Binary Classification

10.1 Task Setup

A binary target was derived: rating_A = 1 if credit.rating == '1' (low risk), else 0. This simplifies the problem to a two-class task, enabling logistic regression and ROC analysis.

10.2 Confusion Matrix & Accuracy

Logistic Regression, Test Set	Pred 0 (Non-A)	Pred 1 (Rating A)
Actual 0 (Non-A)	263	34
Actual 1 (Rating A)	62	34

75.6% Test Accuracy	34/96 Class A Detected (35.4%)	263/297 Non-A Correct (88.6%)
-------------------------------	--	---

While overall accuracy is 75.6%, the model misses 62 of 96 true Class A clients (64.6% false negative rate for Rating A). The classifier is biased toward predicting 'non-A' due to class imbalance, only 24.7% of clients have Rating A.

10.3 Significant Predictors ($p < 0.05$)

Strongly Significant ($p < 0.001$)

Predictor	Coefficient	Interpretation
functionary=1	+2.233 ($p < 2e-16$)	Civil servant status dramatically increases odds of Rating A
re.balanced.overdrawn=1	+2.076 ($p = 0.00092$)	Rebalancing an overdrawn account increases odds of A
FI3O.credit.score=1	+3.770 ($p = 0.00029$)	Positive third-party score is the strongest predictor of A
credit.refused.in.past=1	-1.545 ($p = 6.1e-06$)	Prior credit refusal strongly decreases odds of A

Moderately/Mildly Significant ($p < 0.05$)

Several account balance features achieve significance at the 5% level, including average balance 12, 11, 8, and 7 months ago, and minimum/maximum balances in the most recent 1–3 months. Their direction and magnitude are smaller than the demographic/credit history variables.

Model fit statistics: Null deviance = 1,753 | Residual deviance = 1,220.6 | AIC = 1,544.6, indicating meaningful explanatory power over the null model.

10.4 SVM for Binary Task, 4 Key Features

Using the four most significant logistic regression predictors (functionary, rebalanced_overdrawn, FI3O_credit_score, credit_refused_past), a simplified SVM was fitted with radial kernel (gamma=0.01, cost=10). A randomly selected training customer (seed=456) was predicted as Class B, matching their actual rating.

10.5 ROC Curve Comparison

ROC curves were generated for the logistic regression and tuned SVM on the binary test set (Rating A vs non-A) using the ROCR package. Both models showed curves above the diagonal reference line, confirming better-than-random discriminative ability.

Model	AUC	Strengths
Logistic Regression	Moderate-high AUC	Interpretable coefficients; probability calibration; linear boundary
Tuned SVM (radial)	High AUC	Non-linear boundary; better handling of complex feature interactions

- Key ROC findings:
- Both curves lie above the diagonal, confirming meaningful predictive ability
 - The tuned SVM achieves higher AUC overall, better discrimination across all thresholds
 - Logistic Regression is preferred when interpretability or probability calibration matters
 - SVM is preferred when maximising raw discriminative performance is the goal
 - For a bank deploying this in production, logistic regression's interpretable coefficients may be legally required for credit decision justification

11. Discussion

11.1 Why the Tuned SVM Dominates

The radial basis function kernel implicitly maps features into a high-dimensional space where a linear separator exists for the data. With $\gamma=0.01$ (smooth, wide Gaussian) and $\text{cost}=0.1$ (wide margin, tolerates misclassifications), the model achieves excellent generalisation by avoiding overfitting to Class B's majority. The 1,301 support vectors (vs 1,451 for default) further confirm a cleaner, less noisy boundary.

However, the near-perfect A and B accuracy alongside near-zero C accuracy is suspicious and warrants investigation, it may indicate that the tuning data leakage favoured the majority classes or that the cross-validation folds happened to produce a particularly favourable split.

11.2 Tree-Based Method Limitations

Decision Trees and Random Forests underperform here partly because all features were converted to factors, this eliminates ordinal information from the balance variables (categories 1–5 lose their ordering). Treating balances as ordered or numeric would likely improve tree-based performance.

11.3 The Naïve Bayes Assumption

The independence assumption in Naïve Bayes is severely violated by the 36 account balance columns, max, min, and average balances from adjacent months are highly correlated. This causes the model to multiply similar probabilities redundantly, effectively giving some features far more weight than they deserve. Despite this, 53.2% accuracy is reasonable given the violation.

11.4 Business Implications

For credit decision-making, the cost of false negatives (missing a high-risk client) is typically much higher than false positives (denying credit to a low-risk client). All five classifiers showed poor Class C recall, meaning a bank deploying any of these models would approve a significant proportion of genuinely high-risk clients. Threshold adjustment or cost-sensitive learning should be applied before production deployment.

12. Conclusion

This report evaluated five supervised classification approaches on the credit worthiness dataset. The tuned SVM with radial kernel ($\gamma=0.01$, $\text{cost}=0.1$) achieved the best overall test accuracy at 73.3%, representing a 22.7 percentage point improvement over the default SVM and outperforming all other classifiers. The Decision Tree and Random Forest clustered around 56–58% accuracy, and Naïve Bayes achieved 53.2%.

For the binary Rating A task, Logistic Regression achieved 75.6% accuracy and identified four strongly significant predictors: functionary status, rebalanced overdrawn account, FI3O credit score, and prior credit refusal, all of which align with established credit risk theory. The tuned SVM showed higher AUC on the ROC curve, confirming stronger discriminative ability.

Across all classifiers, Class C (high risk) proved the most difficult to classify correctly, a consistent pattern that highlights the fundamental ambiguity between medium and high-risk financial profiles in this dataset. Addressing this through cost-sensitive learning, SMOTE oversampling, or threshold calibration is the most impactful area for future improvement.

13. Recommendations

13.1 Immediate Improvements

- Apply SMOTE to oversample Class C and Class A in the training set to address class imbalance and improve minority class recall.
- Treat account balance categories as ordered factors (ordered=TRUE in R) to preserve ordinal information for tree-based methods.
- Tune the classification threshold, lower the threshold for Class C prediction to improve high-risk client detection at the cost of some overall accuracy.
- Report precision, recall, and F1-score per class in addition to overall accuracy, these better reflect performance on imbalanced datasets.

13.2 Model Development

- Extend the SVM grid search to include additional kernels (polynomial, sigmoid) and finer cost/gamma grids using cross-validated AUC rather than accuracy as the selection criterion.
- Implement a cost-sensitive Random Forest by assigning higher misclassification costs to Class C errors.
- Engineer derived features from the 12-month balance history (trend slope, volatility, months in negative balance) to provide tree-based models with more informative splits.

13.3 Production Considerations

- Prefer Logistic Regression over SVM if regulatory compliance requires explainable credit decisions (many jurisdictions require lenders to explain rejection reasons).
- Implement probability calibration (Platt scaling or isotonic regression) on the SVM to produce reliable probability estimates alongside class labels.
- Monitor model drift quarterly, credit risk profiles shift with economic conditions, and a model trained in one economic environment may degrade rapidly in another.

14. Limitations & Future Work

14.1 Data Limitations

- Class imbalance (~24.7% A, ~50.2% B, ~25.2% C) was not corrected, biasing all models toward B.
- All features converted to factors, this destroys ordinal and continuous information for tree and SVM models.
- No feature selection or dimensionality reduction was applied, the 36 highly correlated balance columns may be actively harmful for Naïve Bayes.

14.2 Method Limitations

- Single train-test split (80/20, set.seed=123), accuracy estimates have high variance; k-fold cross-validation would provide more reliable measures.
- The tuned SVM's unusual confusion pattern (perfect A and B, near-zero C) warrants further investigation, it may reflect data leakage or an artefact of the specific random seed.
- ROC analysis was performed only for the binary (A vs non-A) task; multi-class ROC (one-vs-rest) for all three classes would give a more complete picture.

14.3 Future Work

- Apply XGBoost or LightGBM as high-performance gradient boosting alternatives, these often outperform all methods evaluated here on tabular financial data.
- Use SHAP (SHapley Additive exPlanations) values to interpret SVM predictions at the individual client level, enabling explainable credit decisions.
- Implement multi-class ROC curves and AUC-ROC for all three credit rating classes across all classifiers.
- Explore time-series modelling of the 12-month balance history (e.g., LSTM) as a feature extractor feeding into the classification layer.

Report: Assignment 2, Credit Worthiness Classification

Author: Ngo Minh Thu Le | Student ID: 8725111

Course: INFO911, Data Mining and Knowledge Discovery

Institution: University of Wollongong | Date: May 2025

Tools: R, rpart, randomForest, e1071 (SVM + Naïve Bayes), ROCR